Indian Language and Code-Mixed Text Identification and Code-Switch Point Analysis Using Character N-Gram TF-IDF, Multinomial Naive Bayes and Softmax Classification

Bandi Thanish Kumar • Pranav Bharadwaj • Manu S Ullikashi • Dewang Sharma
IIIT Dharwad | NLP-13 Project | English, Hindi and Kannada

***Abstract*—This paper presents a transparent, ground-up machine-learning pipeline for identifying the language composition of Romanized and code-mixed text involving English, Hindi and Kannada. The sentence-level task is formulated as seven-class classification: three monolingual classes, three bilingual combinations and one trilingual combination. A character 2–5-gram TF-IDF representation is supplied to two classifiers implemented without scikit-learn: Multinomial Naive Bayes (ManualNB) and a softmax linear classifier trained using stochastic gradient descent (ManualLinear). The corpus contains 3,500 sentences, with 2,779 training, 371 validation and 350 held-out test examples. On the supplied test evaluation, ManualNB correctly classifies 316/350 sentences (90.29%) and obtains macro-F1 0.9029. ManualLinear correctly classifies 313/350 (89.43%) and obtains macro-F1 0.8888. A separate sequential procedure extracts code-switch positions from token-level gold language annotations. Therefore, the reported switch points are annotation-derived rather than predictions from raw text. Four inference examples and both model evaluation outputs are included as evidence.**

***Index Terms*—Code-mixed text, language identification, code-switching, English, Hindi, Kannada, character n-grams, TF-IDF, Multinomial Naive Bayes, softmax regression.**

I. Introduction

Multilingual speakers frequently combine words from different languages in a single message. In Indian digital communication, English, Hindi and Kannada may appear together, often written in Roman script with informal spelling, shortened forms and non-standard grammar. A conventional monolingual language detector can fail when a sentence contains only a short segment from another language. This work studies sentence-level language-composition classification and the identification of token boundaries at which annotated language labels change.

The system has two distinct components. First, a classifier assigns one of seven language-composition labels to a complete sentence. Second, a deterministic switch-point extractor compares adjacent token-level language tags supplied in the dataset. These components must not be conflated: the classifier does not currently generate token-level tags, and the switch-point extractor does not infer tags for previously unannotated text.

Research Objectives

- Classify each sentence into one of seven English/Hindi/Kannada composition categories.
- Build character 2–5-gram TF-IDF features using training data only.
- Implement and compare ManualNB and ManualLinear on the same feature representation.
- Extract switch indices from available token-level language annotations and document the method's scope.

II. Related Work

Prior research has treated language identification in social-media text as a challenging problem because multilingual users may mix languages within a sentence or even a word. Barman et al. examined code-mixing as a language-identification challenge and discussed supervised and sequence-labelling approaches [1]. Work on Indian code-mixed social-media text has also studied word-level language identification [2]. Character n-gram representations are useful for text categorization when spelling and formatting are noisy [3]. GLUECoS provides a benchmark for several code-switched NLP tasks, including language identification [4]. These studies motivate a lightweight and interpretable baseline for the present English–Hindi–Kannada corpus.

III. Dataset and Annotation Protocol

The project dataset is titled "Code-Mixed Real-Life NLP Dataset" and contains 3,500 sentences: 500 examples in each of seven categories. The seven labels are English (E), Hindi (H), Kannada (K), English+Hindi (E+H), English+Kannada (E+K), Kannada+Hindi (K+H), and English+Hindi+Kannada (E+H+K). The source spreadsheet is described as containing sentence text, category, split, token-language tags, switch-point information and generation/provenance metadata. The classifier uses sentence text, category and split; the switch-point parser uses token-language tags.

Dataset Split

The split counts above are transcribed from the supplied dataset-summary screenshot. The test supports are also consistent with the row totals in the two supplied confusion

Category	Train	Validation	Test	Total
English	402	55	43	500
Hindi	398	51	51	500
Kannada	387	60	53	500
English + Hindi	401	50	49	500
English + Kannada	395	50	55	500
Kannada + Hindi	393	53	54	500
English + Hindi + Kannada	403	52	45	500
TOTAL	2779	371	350	3500

matrices. The validation split is kept separate from the final test evaluation.

Preprocessing and Leakage Controls

- Convert text to lowercase and normalize whitespace.
- Pad each sentence with a leading and trailing space before extracting character n-grams.
- Fit the n-gram vocabulary and document-frequency/IDF statistics on the 2,779 training sentences only.
- Ignore n-grams absent from the training vocabulary at test time.
- Keep validation and test examples out of vocabulary fitting and IDF estimation.

The source report identifies standard-library Python components (including math, collections, random and regular expressions) for the core implementation; pandas is used for Excel ingestion. It reports a fixed seed of 42. No pretrained language model is used.

IV. Proposed Methodology

A. End-to-End System Architecture

The sentence-classification path is: raw sentence → normalization and padding → character 2–5-gram extraction → TF-IDF vectorization → ManualNB or ManualLinear → one of seven sentence-level categories. The annotation-analysis path is separate: token/TAG sequence → valid-tag filtering → adjacent-tag comparison → zero-based switch indices and language transitions.

Stage	Input	Output / purpose
Text preparation	Raw sentence	Lowercase, whitespace normalization, boundary padding
Character features	Prepared sentence	Character n-gram counts for n = 2, 3, 4, 5
TF-IDF	Training corpus + sentence	Weighted, L2-normalized feature vector
Sentence classifier	TF-IDF vector	One of seven composition labels
Switch extraction	Token-level gold tags	Indices where consecutive tags differ

B. Core Algorithm 1: Character 2–5-Gram TF-IDF (Five Steps)

1) Normalize: lowercase the input, collapse repeated whitespace and add one boundary space at the beginning and end.
2) Extract n-grams: slide a character window over the padded text and count every substring for $n \in \{2, 3, 4, 5\}$.
3) Learn vocabulary on training data: retain the distinct n-grams observed in the training split and compute each n-gram's document frequency $\mathrm{DF}(g)$.
4) Weight features: multiply the term frequency by smoothed inverse document frequency, $\mathrm{IDF}(g) = \ln[(1 + N)/(1 + \mathrm{DF}(g))] + 1$, where N is the number of training documents.
5) Normalize and transform: L2-normalize the weighted vector. At inference, map only known n-grams into the fixed vocabulary; unseen n-grams are ignored.

$$v(g, d) = \mathrm{TF}(g, d) \times \mathrm{IDF}(g); \quad \mathbf{x}(d) = \mathbf{v}(d)/\|\mathbf{v}(d)\|_2 \quad (1)$$

Character n-grams can retain useful sub-word patterns even when a user spells a word informally or writes an Indic-language word in Roman letters. They do not, by themselves, guarantee correct linguistic interpretation.

C. Core Algorithm 2: Multinomial Naive Bayes (ManualNB) — Five Steps

1) Estimate class priors from the training labels: $P(C_c) = N_c/N_{\mathrm{train}}$.
2) Accumulate non-negative TF-IDF feature weights for every class and every vocabulary feature.
3) Apply additive Laplace smoothing with $\alpha = 1$ to avoid zero feature likelihoods.
4) Convert likelihoods to log probabilities and score each class using the prior plus the weighted sum of feature log-likelihoods.
5) Predict the class with the largest log score; log-space computation avoids numerical underflow.

$$P(f_g \mid C_c) = \frac{\sum_{i \in C_c} x_{i,g} + \alpha}{\sum_{i \in C_c} \sum_k x_{i,k} + \alpha |V|} \quad (2)$$

$$\hat{y} = \arg\max_c \left[\log P(C_c) + \sum_g x_g \log P(f_g \mid C_c) \right] \quad (3)$$

The class prior is calculated from the observed training distribution; it should not be assumed uniform unless all training class counts are equal. The seven training counts in the supplied split are not identical.

V. Core Algorithms and Inference Examples

A. Core Algorithm 3: Softmax Linear Classifier (ManualLinear) — Five Steps

1) Initialize a weight matrix $\mathbf{W}$ and bias vector $\mathbf{b}$ for $K = 7$ classes; the supplied implementation uses zero initialization.
2) For each training vector $\mathbf{x}$, compute one linear score per class: $z_k = b_k + \mathbf{W}_k^\top \mathbf{x}$.
3) Apply softmax to convert scores into class probabilities: $p_k = \exp(z_k)/\sum_m \exp(z_m)$.

Block & Language Category	Source (Generation_Method)	Train Count	Validation Count	Test Count	Total Sentences
Block 1: English	Kaggle	402	55	43	500
Block 2: Hindi	HuggingFace	398	51	51	500
Block 3: Kannada	GitHub	387	60	53	500
Block 4: English + Hindi	AI4Bharat	401	50	49	500
Block 5: English + Kannada	Kaggle	395	50	55	500
Block 6: Kannada + Hindi	GitHub	393	53	54	500
Block 7: English + Hindi + Kannada	HuggingFace	403	52	45	500
TOTAL (All 3,500 Rows)		**2779**	**371**	**350**	**3500**
Description :					
Conversational dataset with English, Hindi, and Kannada sentences, along with code-mixed pairs and tri-lingual scripts designed for real-life dialog models and code-switched NLP benchmarks.					

Fig. 1. Dataset provenance and train/validation/test distribution (source screenshot reproduced without editing).

4) Compute the categorical cross-entropy gradient using the difference between predicted probability and the one-hot target.
5) Update $\mathbf{W}$ and $\mathbf{b}$ using per-sample stochastic gradient descent. The reported configuration uses learning rate $\eta = 0.15$, 35 epochs, fixed sample order and no regularization.

$$P(y = k \mid \mathbf{x}) = \frac{\exp(b_k + \mathbf{W}_k^\top \mathbf{x})}{\sum_m \exp(b_m + \mathbf{W}_m^\top \mathbf{x})} \tag{4}$$

$$\begin{aligned} W_{k,j} &\leftarrow W_{k,j} - \eta\,(p_k - \delta_{k,y})\,x_j \;; \\ b_k &\leftarrow b_k - \eta\,(p_k - \delta_{k,y}) \end{aligned} \tag{5}$$

B. Core Algorithm 4: Sequential Code-Switch Point Extractor — Five Steps

1) Read the sentence's token/TAG annotation and split it into the token sequence.
2) Extract the tag after the final slash and retain only valid tags E, H or K; invalid/punctuation entries are discarded.
3) Compare each adjacent pair of tags, from the second retained token to the last.
4) If the current tag differs from the previous tag, record the current token's zero-based index and the transition (previous $\rightarrow$ current).
5) Report the number of switches and switch density $\rho = S_{\text{num}}/M$, where M is the number of retained tagged tokens.

$$\text{Switch}(i) = 1 \text{ if } \text{tag}_i \neq \text{tag}_{i-1}, \text{ for } i = 1, \ldots, M-1 \tag{6}$$

Important scope statement: this algorithm operates on gold token-level annotations already present in the dataset. It is a deterministic annotation-processing routine, not a trained token-level language-identification model. Predicting switch points from a raw unseen sentence would require a separate token-level tagger, which is outside the implemented system.

C. Inference Example 1 — Kannada Monolingual Sentence

D. Inference Example 2 — Kannada + English

VI. Additional Qualitative Inference Examples

A. Inference Example 3 — Hindi + English

The screenshot shows a sentence beginning with Romanized Hindi and continuing in English. The output lists token-level labels and identifies the first English token as a switch location. These are the system's displayed predictions/annotations for this example; the screenshot alone does not establish independent linguistic ground truth.

```
Input Sentence: "Naanu naale raje togottulliddini"

Predicted Category: [Kannada]

Word-by-Word Language Tagging:
  - 'Naanu': Kannada
  - 'naale': Kannada
  - 'raje': Kannada
  - 'togottulliddini': Kannada

Language Switch Points:
  * No language switch points detected. Monolingual sentence.

Trilingual Analysis:
  * Languages present in sentence: Kannada
```

Fig. 2. Example output: monolingual Kannada sentence, word-level tags and no detected language switch.

```
Input Sentence: "Ivallu weather channagide walking hogana "

Predicted Category: [English + Kannada]

Word-by-Word Language Tagging:
  - 'Ivallu': Kannada
  - 'weather': English
  - 'channagide': Kannada
  - 'walking': English
  - 'hogana': Kannada

Language Switch Points:
  * Language change at Word Index 1 ('weather'): Switched from [Kannada] -> [English]
  * Language change at Word Index 2 ('channagide'): Switched from [English] -> [Kannada]
  * Language change at Word Index 3 ('walking'): Switched from [Kannada] -> [English]
  * Language change at Word Index 4 ('hogana'): Switched from [English] -> [Kannada]

Trilingual Analysis:
  * Languages present in sentence: English, Kannada
```

Fig. 3. Example output: Kannada–English code-mixed sentence with token tags and switch transitions.

B. Inference Example 4 — English + Hindi + Kannada

This example demonstrates the intended three-language composition case. The output reports English, Hindi and Kannada as present and lists four language transitions. The token labels and switch indices are reproduced as shown in the submitted output.

C. Interpretation of the Examples

- A monolingual sentence is expected to contain one distinct language label and zero adjacent-tag changes.

```
Input Sentence: "Aaj ghoomne chalte hain we will eat dosa"

Predicted Category: [English + Hindi]

Word-by-Word Language Tagging:
  - 'Aaj': Hindi
  - 'ghoomne': Hindi
  - 'chalte': Hindi
  - 'hain': Hindi
  - 'we': English
  - 'will': English
  - 'eat': English
  - 'dosa': English

Language Switch Points:
  * Language change at Word Index 4 ('we'): Switched from [Hindi] -> [English]

Trilingual Analysis:
  * Languages present in sentence: English, Hindi
```

Fig. 4. Example output: Hindi–English sentence; the displayed tags show the transition from Hindi to English.

```
Input Sentence: "It is raining heavily isliye main auto lelunga mattu nanna kayabedi"

Predicted Category: [English + Hindi + Kannada]

Word-by-Word Language Tagging:
  - 'It': English
  - 'is': English
  - 'raining': English
  - 'heavily': English
  - 'isliye': Hindi
  - 'main': Hindi
  - 'auto': English
  - 'lelunga': Hindi
  - 'mattu': Kannada
  - 'nanna': Kannada
  - 'kayabedi': Kannada

Language Switch Points:
  * Language change at Word Index 4 ('isliye'): Switched from [English] -> [Hindi]
  * Language change at Word Index 6 ('auto'): Switched from [Hindi] -> [English]
  * Language change at Word Index 7 ('lelunga'): Switched from [English] -> [Hindi]
  * Language change at Word Index 8 ('mattu'): Switched from [Hindi] -> [Kannada]

Trilingual Analysis:
  >>> SUCCESS: Sentence contains ALL THREE languages (English + Hindi + Kannada)! <<<
```

Fig. 5. Example output: trilingual sentence containing English, Hindi and Kannada tokens, with the displayed switch positions.

- A bilingual or trilingual sentence can contain multiple switch points; the count depends on the order of token-level tags, not merely on the number of languages present.
- A sentence-level class label summarizes which languages are present, whereas switch indices describe where the sequence changes from one language tag to another.
- Romanized spellings, loanwords, named entities and short tokens can be ambiguous. A displayed tag should therefore be interpreted as a model/dataset label rather than a definitive linguistic judgement without manual annotation review.

VII. Experimental Setup and Evaluation Metrics

A. Experimental Protocol

Both classifiers use the same character 2–5-gram TF-IDF feature representation. The vocabulary and IDF values are learned from the training partition only. Training uses 2,779 sentences; the validation partition contains 371 sentences; final evaluation is reported on 350 held-out test sentences. The test set contains 50 sentences per class in the original balanced design only if the split was stratified equally; the supplied test-support matrix instead shows the actual per-class supports [43, 49, 45, 55, 51, 53, 54], which sum to 350. Accordingly, the report uses these observed supports for per-class interpretation.

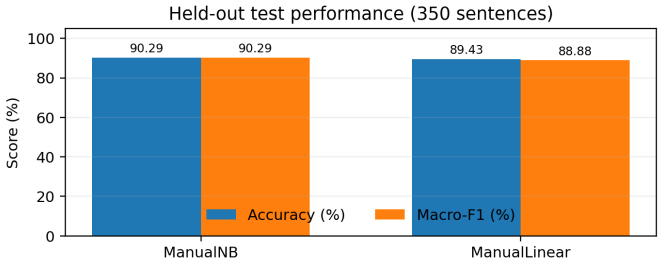


Fig. 6. Comparison of accuracy and macro-F1 from the supplied held-out test results.

B. Evaluation Metrics (Five Key Measures)

1) Accuracy: fraction of all test sentences classified correctly; it gives a single overall measure.
2) Precision for class c: among all predictions assigned to c, the proportion that truly belong to c; high precision means fewer false positives for that class.
3) Recall for class c: among all true examples of c, the proportion recovered by the classifier; high recall means fewer false negatives.
4) F1-score for class c: harmonic mean of precision and recall, balancing the two measures.
5) Macro-F1: arithmetic mean of the seven class-wise F1 scores, giving every category equal weight regardless of support.

$$\text{Accuracy} = \text{correct predictions}/\text{total test examples} \tag{7}$$

$$\text{Precision}_c = \frac{\text{TP}_c}{\text{TP}_c + \text{FP}_c}; \quad \text{Recall}_c = \frac{\text{TP}_c}{\text{TP}_c + \text{FN}_c} \tag{8}$$

$$\text{F1}_c = \frac{2 \times \text{Precision}_c \times \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c} \tag{9}$$

$$\text{Macro-F1} = \frac{1}{7} \sum_c \text{F1}_c \tag{10}$$

C. Overall Model Comparison

Model	Correct / Test	Errors	Accuracy	Macro-F1
ManualNB	316 / 350	34	90.29%	0.9029
ManualLinear	313 / 350	37	89.43%	0.8888
Difference	3 more correct	3 fewer errors	+0.86 percentage points	+0.0141

The observed accuracy difference is 3 test sentences (0.86 percentage points). The supplied short paper characterizes this difference as within approximate sampling uncertainty. The experiment is a single fixed split and does not establish statistical superiority across repeated samples.

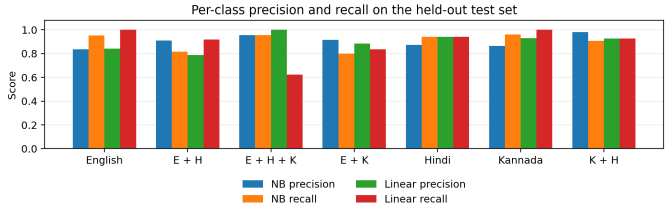


Fig. 7. Class-wise precision and recall for ManualNB and ManualLinear on the 350-sentence held-out test set.

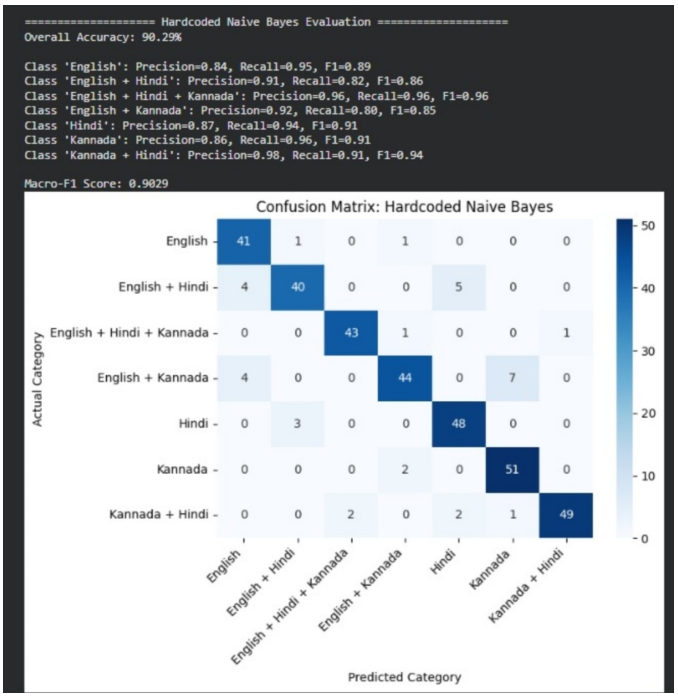


Fig. 8. ManualNB evaluation screenshot: per-class precision/recall/F1, macro-F1 and confusion matrix.

D. Per-Class Precision–Recall Comparison

The graph highlights the trade-off between false-positive and false-negative errors across categories. ManualLinear reaches 1.00 recall for English and Kannada, but its trilingual recall is 0.62; ManualNB has more balanced trilingual precision and recall (both 0.96). These values are computed from the supplied confusion matrices.

VIII. Detailed Results and Confusion-Matrix Analysis

A. Per-Class Test Metrics

The table reproduces the two-decimal per-class values shown in the supplied evaluation outputs and the two-page paper. Macro-F1 and accuracy are computed over the complete seven-class test evaluation.

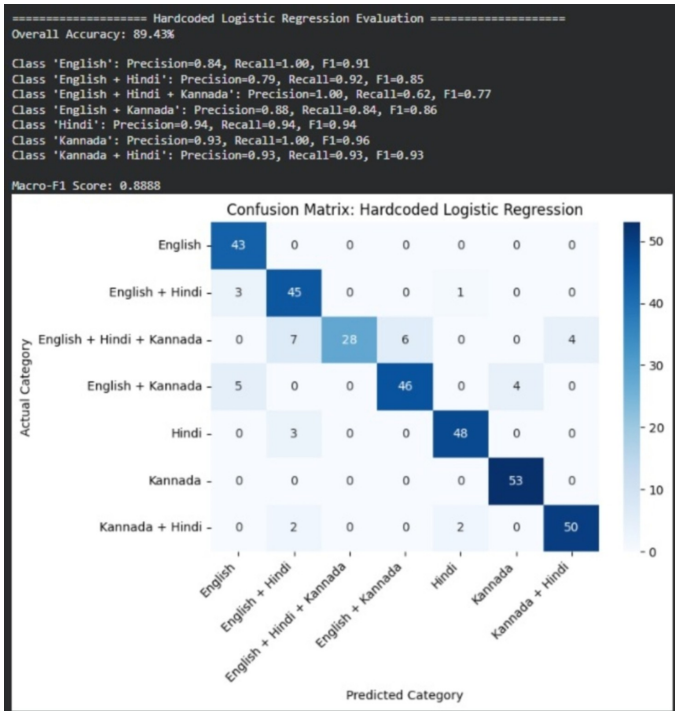


Fig. 9. ManualLinear evaluation screenshot: per-class precision/recall/F1, macro-F1 and confusion matrix.

B. ManualNB Evaluation Output and Confusion Matrix

C. ManualLinear Evaluation Output and Confusion Matrix

IX. Error Analysis, Limitations and Future Work

A. Error Analysis

The errors are concentrated in code-mixed classes rather than being uniformly distributed across all labels. For ManualNB, the largest confusion group is English+Kannada: 11 of its 49 test sentences are misclassified, with 7 labelled Kannada and 4 labelled English. English+Hindi is the next major source, with 9 of 49 sentences misclassified (5 as Hindi and 4 as English). In both cases, the model assigns a bilingual sentence to one of its constituent monolingual classes. This pattern is consistent with a possible dominant-language effect: when one language contributes more recognizable character n-grams than the other, the sentence-level representation may favour that language. This is a plausible interpretation of the observed confusions, not a cause directly tested by the experiment.

ManualLinear makes 37 errors overall and shows its clearest weakness on trilingual input. It correctly classifies 28 of 45 English+Hindi+Kannada test sentences; the remaining 17 are predicted as English+Hindi (7), English+Kannada (6), or Kannada+Hindi (4). Consequently, its trilingual precision is 1.00 while recall is 0.62: predictions assigned to the trilingual class are correct in this test set, but many actual trilingual sentences are missed. ManualNB performs substantially better on this class, identifying 43 of 45 sentences (recall 0.96). Across all seven classes, ManualNB also has fewer errors (34 versus 37) and a higher macro-F1 score (0.9029 versus

Class	Support	NB Precision	NB Recall	NB F1	Linear Precision	Linear Recall	Linear F1
English	43	0.84	0.95	0.89	0.84	1.00	0.91
English + Hindi	49	0.91	0.82	0.86	0.79	0.92	0.85
English + Hindi + Kannada	45	0.96	0.96	0.96	1.00	0.62	0.77
English + Kannada	55	0.92	0.80	0.85	0.88	0.84	0.86
Hindi	51	0.87	0.94	0.91	0.94	0.94	0.94
Kannada	53	0.86	0.96	0.91	0.93	1.00	0.96
Kannada + Hindi	54	0.98	0.91	0.94	0.93	0.93	0.93

0.8888). A likely challenge for character n-gram models is that Romanized spelling variation, shared or borrowed words, short language segments, and changes in language order can make different code-mix categories look similar. The present results do not isolate the contribution of each factor; token-level ablation and a larger, independently collected test set would be needed to verify them. The confusion matrices therefore suggest that detecting the presence of several languages—especially all three in one sentence—is harder than recognizing a dominant monolingual pattern.

B. Limitations

- The results come from one fixed train/validation/test split and one reported run; repeated splits and confidence intervals are not supplied.
- The switch-point routine reads gold token annotations. It is not a learned token-level predictor and its switch-point accuracy is not evaluated.
- The dataset uses Romanized text and informal spelling; script-specific performance and per-token error patterns are not separately reported.
- The source/provenance field names data platforms or repositories, but the screenshots and paper do not independently document licensing, exact source URLs, or sentence-level provenance verification.
- No neural or pretrained multilingual model is evaluated; conclusions apply only to the two implemented classical baselines and this dataset.

C. Future Work

- Develop a supervised token-level language identifier (e.g., a sequence-labeling model) to predict E/H/K tags from raw text.
- Evaluate predicted switch points against manually verified token-level gold labels using boundary precision, recall and F1.
- Repeat experiments across multiple stratified random splits and report mean, standard deviation and confidence intervals.
- Inspect ambiguous Romanized tokens, named entities and loanwords; add annotation guidelines and independent review.
- Compare against transparent baselines such as character n-gram logistic regression and simple word/character dictionary methods.

X. Conclusion

This work implements a reproducible, interpretable baseline for seven-way sentence-level identification of English, Hindi, Kannada and their combinations. Character 2–5-gram TF-IDF features support both classifiers. ManualNB achieves 90.29% test accuracy and 0.9029 macro-F1, while ManualLinear achieves 89.43% and 0.8888, respectively. The difference corresponds to three test sentences and should not be overstated. The separate switch-point routine provides deterministic boundary extraction from gold token annotations; raw-text token prediction remains future work. The included outputs document the implemented pipeline and its observed behaviour.

References

[1] U. Barman, A. Das, J. Wagner, and J. Foster, "Code Mixing: A Challenge for Language Identification in the Language of Social Media," Proc. First Workshop on Computational Approaches to Code Switching, 2014, pp. 13–23.

[2] A. Das and B. Gambäck, "Identifying Languages at the Word Level in Code-Mixed Indian Social Media Text," Proc. 11th International Conference on Natural Language Processing (ICON), 2014, pp. 378–387.

[3] W. B. Cavnar and J. M. Trenkle, "N-Gram-Based Text Categorization," Proc. 3rd Annual Symposium on Document Analysis and Information Retrieval, 1994, pp. 161–175.

[4] S. Khanuja et al., "GLUECoS: An Evaluation Benchmark for Code-Switched NLP," Proc. 58th Annual Meeting of the Association for Computational Linguistics, 2020, pp. 3575–3585.

[5] A. McCallum and K. Nigam, "A Comparison of Event Models for Naive Bayes Text Classification," AAAI-98 Workshop on Learning for Text Categorization, 1998.

[6] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.